

The two orchestrators, explained with stick figures

Phase 2b — what we built, what we measured, and where the proof is

Team 23UG005 · Open5GS stack, 100 UEs · 8 October 2026

eMBB road — big, can wait



URLLC lane — small, must be fast



mMTC path — many tiny sensor reports ● ● ● ● ● ● ● ● ● ● ● ● ●



Slice orchestrator
= the traffic cop:
who goes on which
road, and who first

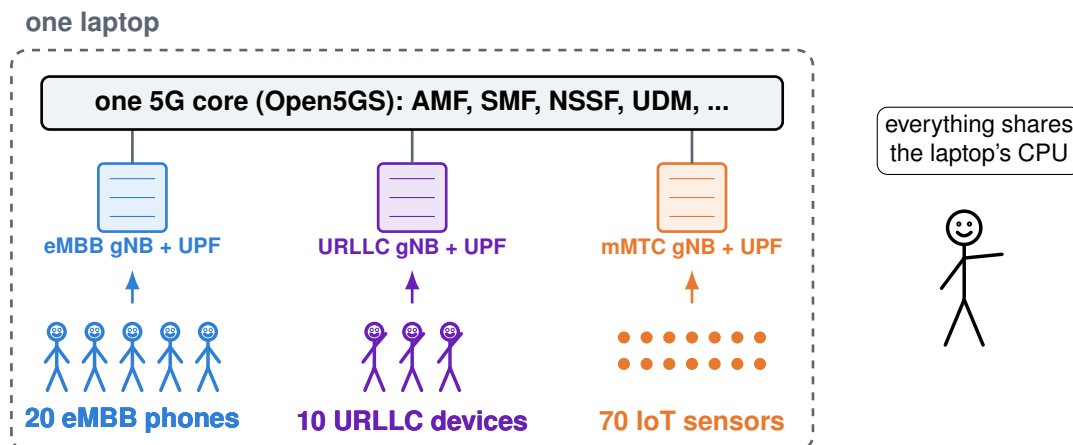


Network orchestrator
= the road builder:
opens and closes
whole roads

In one paragraph. Our 5G network has three slices, like three roads. The **slice orchestrator** is the traffic cop: it decides which road each request uses, lets VIPs (critical devices) go first, and slows down the big road when it starts hurting the fast lane. The **network orchestrator** is the road builder: it opens a whole road when someone needs it, closes it when nobody does, checks every step, and undoes its work if something goes wrong. Everything below was measured on our running 100-UE stack, and every number has a file behind it.

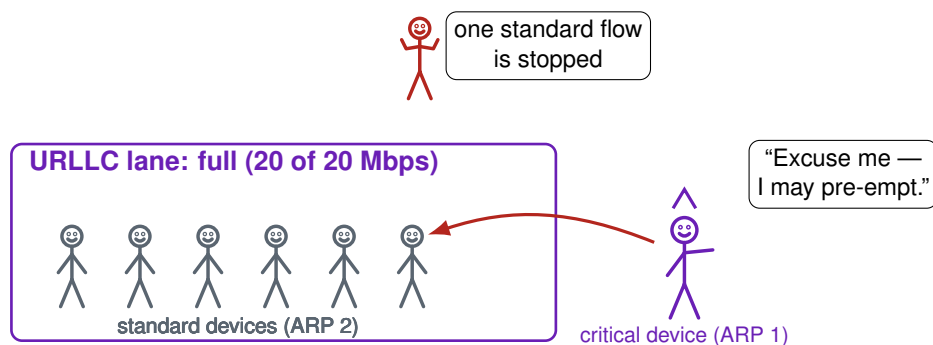
Contents 1 The setup 2 Traffic cop: priority 3 Traffic cop: the feedback controller 4 Traffic cop: planning 5 Road builder: off and on 6 Road builder: rollback 7 Road builder: on its own (closed loop) 8 Road builder: compute 9 Which algorithm is best? 10 The photos: one live run 11 Things that went wrong 12 Where every proof lives 13 Sources

1 The setup: 100 phones and sensors, three slices



Each slice has its own radio (gNB), its own user-plane (UPF) and its own data network. What they *share* is the laptop's CPU. Remember that — it turns out to matter in section 3.

2 Traffic cop: priority inside URLLC (ARP)

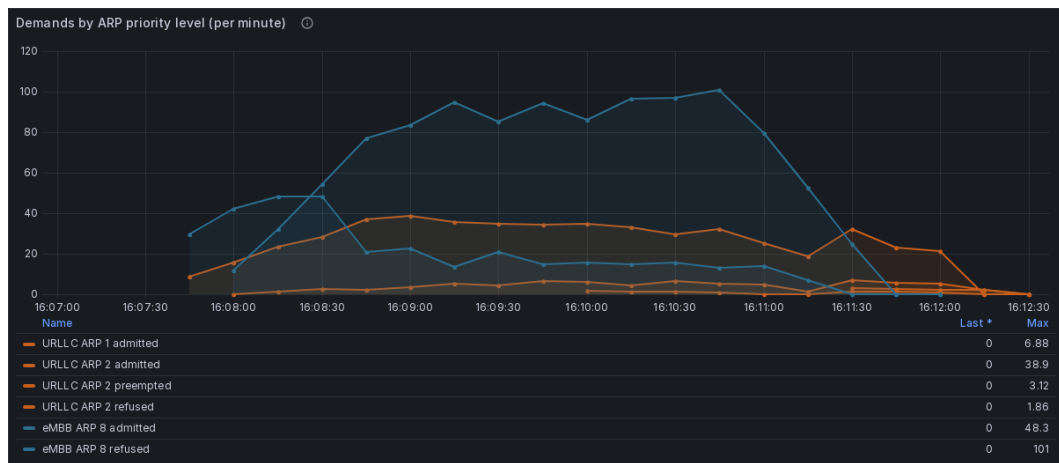


3GPP has a rule for exactly this: **ARP** (Allocation and Retention Priority). We gave 3 of the 10 URLLC devices ARP 1 ("critical, may pre-empt") and 7 of them ARP 2 ("standard, may be pre-empted"). The rule is stored in each device's subscription in the core.

What we did	What happened
20 standard demands	all admitted — the lane is full (20/20 Mbps)
1 more standard demand	refused : an equal priority cannot push anyone out
5 critical demands	all admitted , each by stopping exactly one standard flow
the flows afterwards	critical 5/5 got their full rate; the 15 standard flows left also got theirs

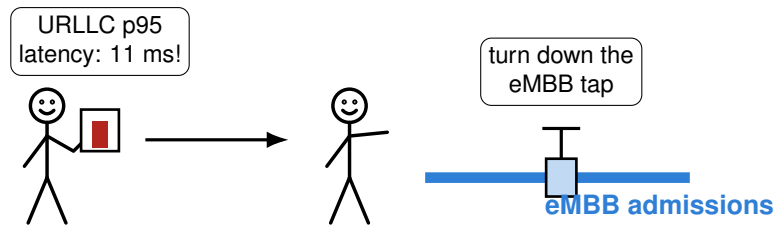
We ran it twice and got the same result both times.

Proof: docs/evidence/open5gs-orchestrator-dynamic/1-priority.txt



Grafana, from the live run in section 10: demands per minute by ARP level. “URLLC ARP 1 admitted” is the critical devices; “URLLC ARP 2 preempted” is the standard flows they pushed out.

3 Traffic cop: the feedback controller



Every 5 seconds the controller looks at URLLC’s latency (the 95th percentile over the last 10 s). If it goes above 8 ms (80% of the 10 ms budget), it **cuts** how much eMBB traffic is admitted. When things are calm again and eMBB wants more, it **raises** the limit a little. That is the same “cut fast, grow slowly” rule TCP uses (AIMD).

Why eMBB? We measured it. Each kind of traffic alone, controller off:

Load (120 s)	URLLC p95	URLLC p99	host load
nothing (idle)	2.5 ms	5.4 ms	2.3
URLLC demand only	2.8 ms	5.3 ms	5.0
eMBB demand only	8.9 ms	17.5 ms	36.9

Proof: docs/evidence/open5gs-orchestrator-dynamic/4-diagnosis.txt

URLLC’s own traffic does not slow URLLC down. eMBB’s does — through the CPU they share.

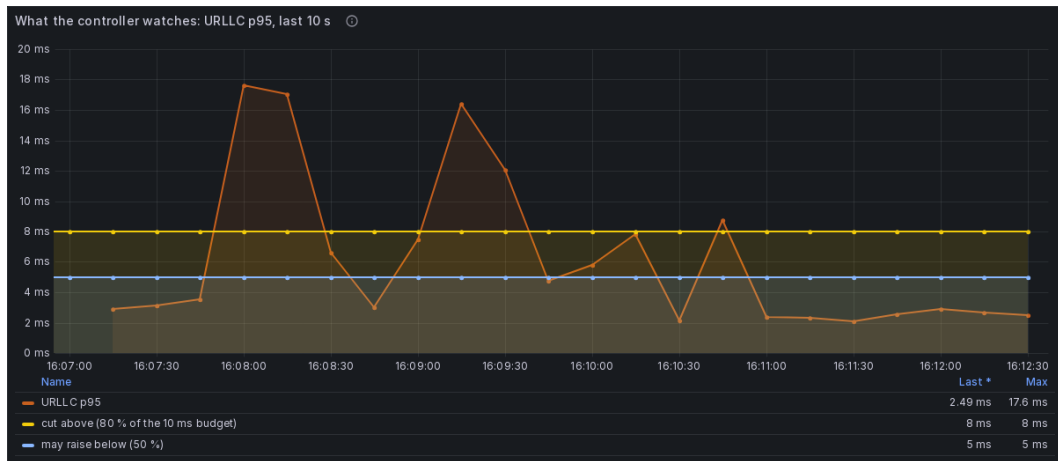
The result — same demand, controller off vs on (180 s each):

	controller off	controller on
eMBB flows that got their full rate	68 %	100 %
URLLC latency mean / p95 / p99	5.1 / 20.5 / 55.4 ms	3.2 / 12.5 / 34.0 ms
URLLC flows that got their full rate	100 %	100 %
eMBB data actually delivered	266 Mbps	109 Mbps

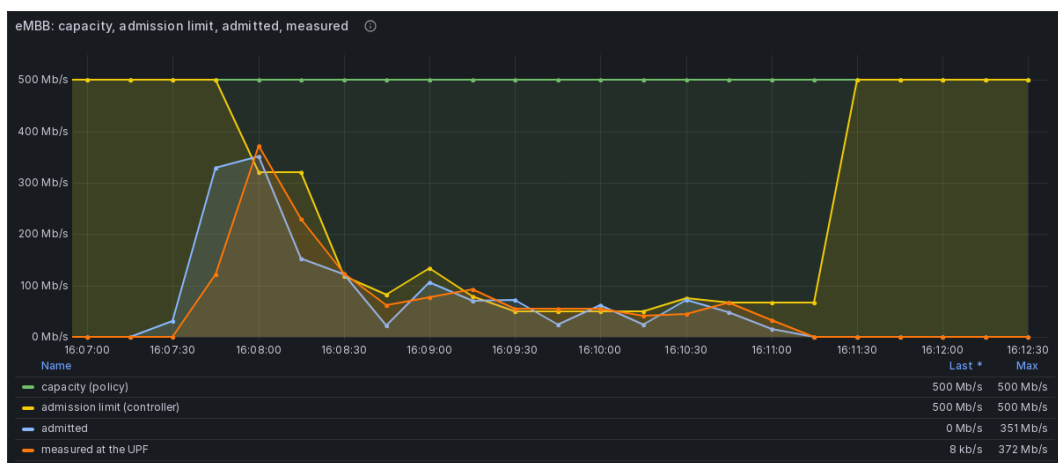
Proof: docs/evidence/open5gs-orchestrator-dynamic/2-comparison.txt

Honest note. The controller makes every admitted eMBB flow work and cuts URLLC’s slow tail by about 40 %, but it does *not* keep URLLC under the 10 ms target the whole time on this laptop, and it costs eMBB volume. Much of that “266 Mbps” was broken flows, though.

Update: this section describes the first controller (AIMD). We later raced it against other methods, and a PI controller did better on every measure: 0 % of the time over budget, with as much eMBB as no control. It is the default now (section 9).

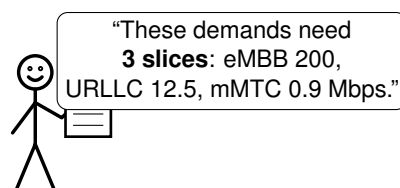


What the controller watches (live run, section 10): URLLC p95 over the last 10 s. Each time it climbs over the yellow 8 ms line, the controller cuts eMBB, and it comes back down.



eMBB in the same run: the policy capacity (green, 500 Mbps) and the controller's admission limit (yellow) falling from 500 to 50–130 Mbps under load. The jump back to 500 at 16:11:20 is the orchestrator being restarted for the priority test, not the controller.

4 Traffic cop: planning — how many roads do we need?



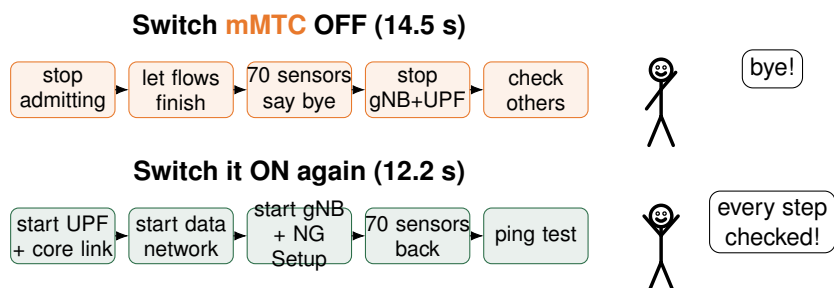
/plan looks at a set of demands (or what it actually saw in the last few minutes) and says which slices are needed and how much capacity each needs. Every kind of traffic goes on the cheapest road that is still good enough.

Demand	slices needed	capacity needed
Phase 2b mix: 10 control, 20 video, 70 sensors	3	eMBB 200, URLLC 12.5, mMTC 0.88 Mbps
only video and files (can wait)	1	eMBB 487.5 Mbps
what the orchestrator saw in a 180 s run	2	eMBB 363, URLLC 13.5 Mbps

Proof: docs/evidence/open5gs-orchestrator-dynamic/3-planning.txt

The road builder (next sections) is the one who *acts* on this answer.

5 Road builder: switching a slice off and on



	mMTC on	mMTC off
mMTC sessions in the core	70	0
eMBB / URLLC devices still online	20 / 10	20 / 10
UE container CPU / memory	211 % / 200 MiB	40 % / 75 MiB
a sensor asks for service	admitted	refused: "mMTC is not active"

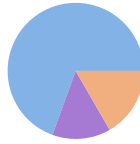
Proof: docs/evidence/open5gs-network-orchestrator/1-lifecycle.txt

While all this happened, URLLC's latency probes lost **0 of 63 891** packets (p95 2.8 ms): switching one slice did not disturb the others.



Grafana, live run (section 10): each slice serving (1) or dormant (0). mMTC goes off at 16:07:56 and back on at 16:09:30; eMBB and URLLC stay at 1 (their lines overlap).

8 Road builder: how much CPU each slice gets



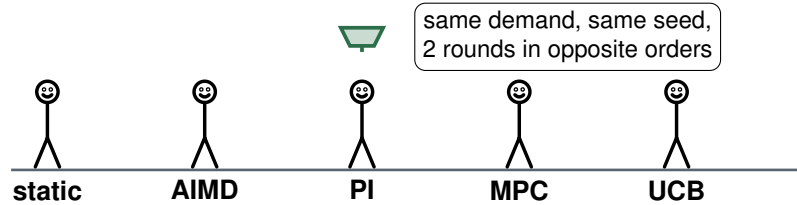
"eMBB, you get
only 1 CPU now."

CPU for eMBB's gNB + UPF	eMBB carried	URLLC p99 meanwhile
unlimited	366 Mbps	11.0 ms
1 CPU	121 Mbps	10.4 ms
0.5 CPU	50 Mbps	8.1 ms

Proof: [docs/evidence/open5gs-network-orchestrator/4-compute.txt](#)

The knob works: eMBB's capacity follows its CPU. URLLC improves only a little, so whether to use it is a policy decision. By default eMBB is unlimited.

9 Which algorithm is best? We raced them



For every decision the orchestrators make, we looked up the main methods in the research literature, implemented the strongest ones, and raced them under identical conditions.

- **Live** on our stack, where the network itself matters: the latency controller.
- **In a simulator** running our real decision code, where it does not: admission, sizing and switching, over tens of thousands of demands.

The full table, with references, is in `docs/algorithm-comparison.md`.

The latency controller: live, 300 s each, mean of 2 rounds

Control law	URLLC over 10 ms	URLLC p99	eMBB flows met	eMBB delivered
static (no control)	71 % of the time	52.7 ms	60 %	260 Mbps
AIMD (our old default, like TCP)	20.5 %	21.0 ms	87 %	170 Mbps
PI (like PIE, RFC 8033)	0.0 %	13.0 ms	100 %	261 Mbps
MPC (model-predictive)	9.0 %	15.5 ms	95 %	154 Mbps
UCB bandit (online learning)	9.0 %	17.2 ms	97 %	163 Mbps

Proof: `docs/evidence/open5gs-algorithm-comparison/controller/comparison.md`

PI kept URLLC inside its budget the *whole* time in both rounds, *and* carried as much eMBB as no control at all. It is now the default.

Every decision: the winner

Decision	Best on our stack	Why (measured)
how much eMBB to admit	PI (was AIMD)	above
who gets in when it is full	trunk reservation (was first-come)	critical demands blocked 21 % → 2 %, most value carried (simulation)
how much capacity a slice needs	Kaufman–Roberts (was mean × 1.25)	meets a 1 % blocking target; the old rule gave 3–6 % (simulation)
when to switch a slice on	predictive + reactive (was reactive)	periodic IoT demand refused 42 % → 4–5 % (simulation)
which flow to pre-empt	unchanged (the 3GPP ARP rule)	the alternatives pick the same flows when all flows are the same size, as ours are

Proof: `docs/evidence/open5gs-algorithm-comparison/simulation/`

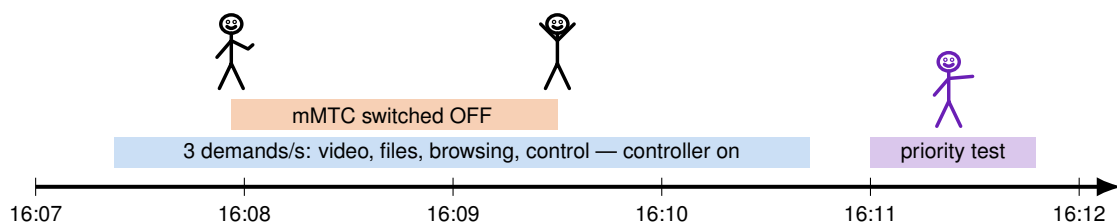
What about deep reinforcement learning? It is the most popular method in recent slicing papers, and they report it as best, in simulation. We did *not* implement it.

- On our testbed one decision takes 5 s of real time, so training would need many hours of live load.
- While learning, it would deliberately break URLLC's latency budget.
- We could not explain the policy it learned.

The fair next step: train it offline on a model fitted to these measurements, then race it against PI the same way.

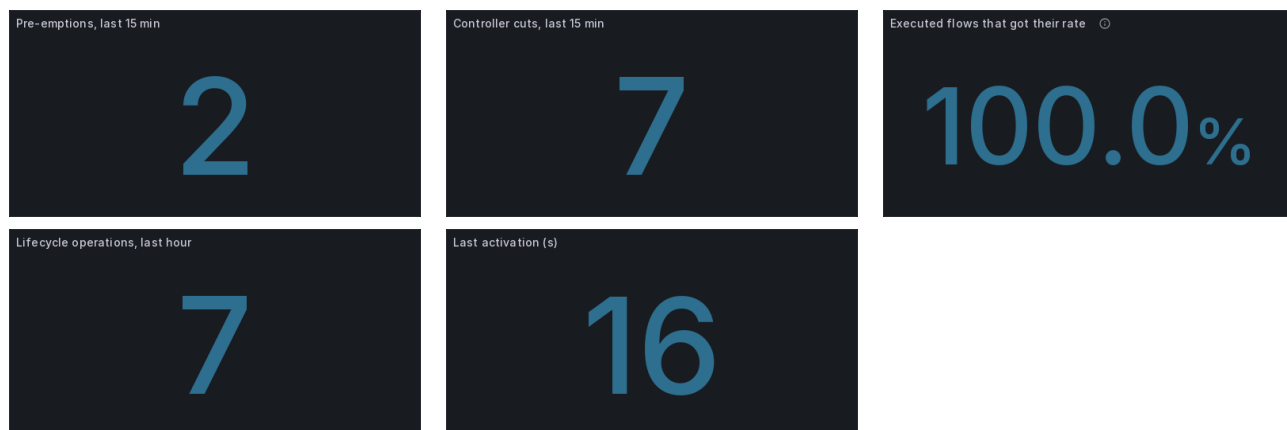
10 The photos: one live run, minute by minute

All the Grafana screenshots in this document come from **one scripted 5-minute run** on 8 October (16:07–16:12 UTC), so they show the same events. The numbers in the tables above come from the separate, longer evidence runs; this run is here so you can see it happen.

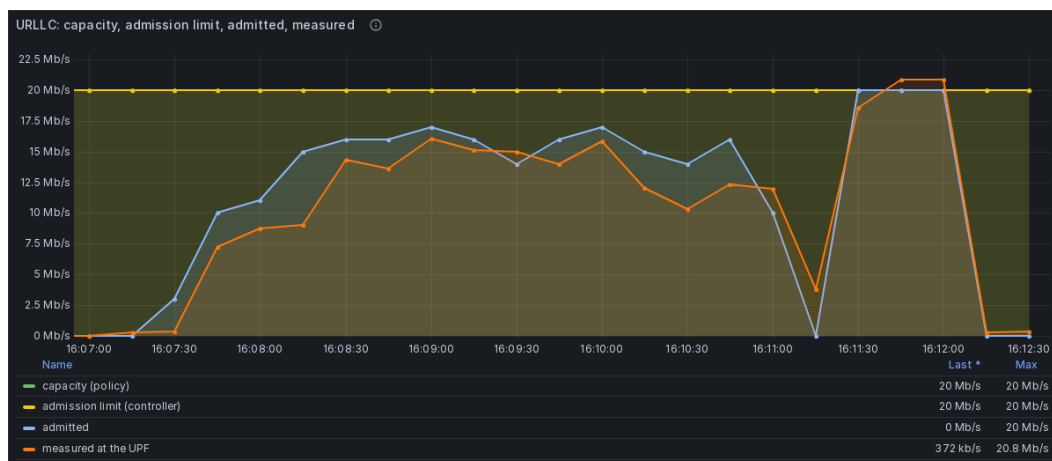


Time (UTC) What happened

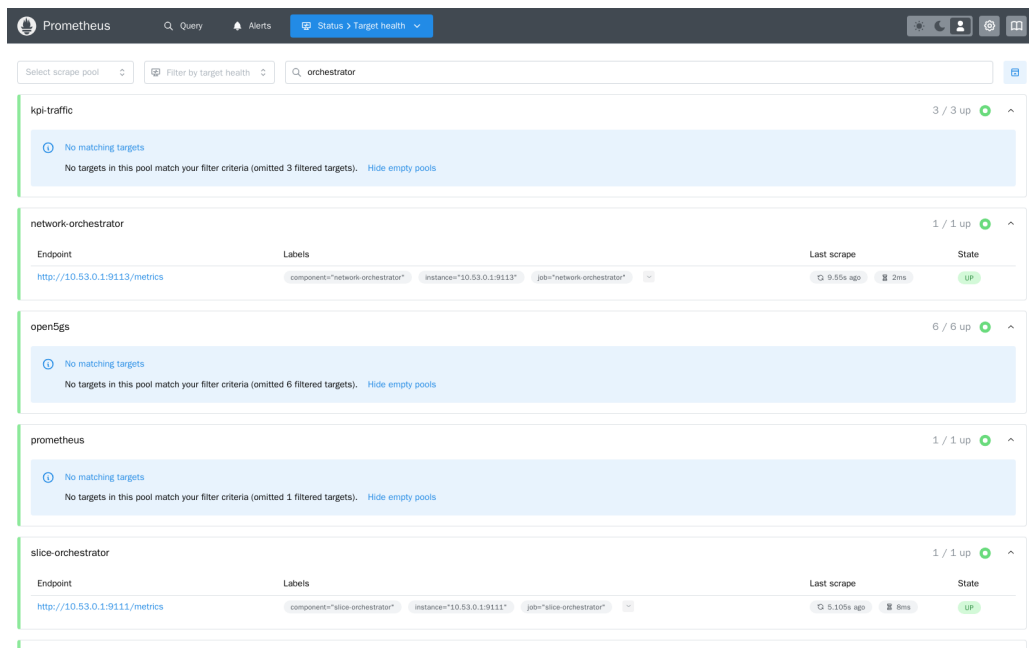
16:07:25	demand starts; the controller soon cuts eMBB's limit (500 → 320 → 120 Mbps) as URLLC p95 jumps to 17.6 ms
16:07:56	road builder switches mMTC off: done in 31.7 s (it waited for the busy host)
16:09:30	mMTC on again: done in 16.0 s
16:10:45	demand ends: 215 admitted, 265 refused (the controller and the dormant slice refused them)
16:11:20	priority test: 20 standard URLLC demands, then 5 critical ones



Counters from the same run: pre-emptions, controller cuts, executed flows that got their rate, lifecycle operations, and the last activation's time in seconds.

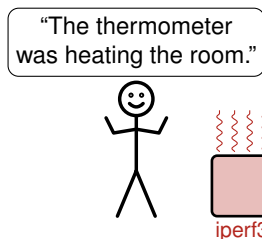


URLLC in the same run: what was admitted and measured on the fast lane (capacity 20 Mbps).



Prometheus scraping both orchestrators: `network-orchestrator (:9113)` and `slice-orchestrator (:9111)` are UP.

11 Things that went wrong (and what we learned)



Problem	What it did, and the fix
Our traffic tool was a CPU hog	iperf3's UDP sender used 45–90 % of a CPU core <i>per flow</i> , at any speed. The load it put on the laptop is what made URLLC slow, so the first two versions of the controller looked broken and even blamed the wrong slice. Fix: our own light flow tool (1–10 % of a core). The bad runs are kept, clearly marked as superseded.
Leaked processes	A timed-out flow left its sender running for 5 minutes at 90 % CPU. Fix: every flow runs under a hard kill timer.
Port clash	Flows were given the same port as our KPI collector, so 7 of 26 failed instantly. Fix: flows use ports from 5410.
Lost commands to devices	The UE command tool sent "deregister" to stale entries of old processes, so 38 of 70 never arrived. Fix: stale entries are cleaned by checking which ports are really in use.
Sessions left in the core	Devices were killed before their "I'm leaving" message went out. Fix: let each device switch itself off first. Now the core frees 69–70 of 70.
"Unlimited CPU" did nothing	<code>docker update -cpus 0</code> is silently ignored, but we reported success. Fix: unlimited = all 12 cores, and every change is read back.
Kubernetes would have broken	The UE pod would not have found its new helper script. Fixed in the generated manifests, not yet run on minikube.
Stale sessions under heavy load	In the live run (section 10), with the host busy, 11 of 70 sensors stopped before their "leaving" message reached the core, so the core kept counting them while mMTC was off. They clear when the sensors come back. Not fixed: switching off now reports it as a clear WARNING instead of a quiet detail.

The lesson for all of us: before trusting a measurement, check what the measuring tool itself costs. Every run now records the host's load and any leftover processes.

12 Where every proof lives

Folder / file	What is inside
docs/evidence/ open5gs-orchestrator-dynamic/	priority, controller on/off, planning, diagnosis — every flow and every controller decision as JSON
docs/evidence/ open5gs-network-orchestrator/	off/on, rollback, closed loop, compute — every operation with its step timings
docs/evidence/open5gs-orchestrator-dynamic/ superseded-iperf3-load/	the runs spoiled by iperf3 — kept on purpose, do not quote them
docs/screenshots/open5gs/10-*.png to 21-*.png	the Grafana screenshots in this document
docs/adr/ADR-016-dynamic-slice-allocation.md	the traffic cop's design decisions
docs/adr/ADR-017-network-orchestrator.md	the road builder's design decisions

To see it yourself (on the lab machine, stack running):

make o5gs-network	state of every slice
python3 tools/network-orchestrator/netorch.py deactivate mMTC	switch a slice off
python3 tools/network-orchestrator/netorch.py activate mMTC	...and on again
make o5gs-dynamic	re-run the traffic cop experiments (≈ 25 min)
make o5gs-network-demo	re-run the road builder experiments (≈ 16 min)
http://localhost:3001 (Grafana)	rows "Dynamic allocation" and "Network orchestrator"

Still provisional — to confirm with our guide: 3 critical / 7 standard URLLC devices; the controller's 8 ms / 5 ms lines; mMTC as the only on-demand slice; 120 s idle time; 150 s activation deadline.

13 Sources

Standards

- [1] 3GPP TS 23.501, *System architecture for the 5G System*, §5.7.2.2 (ARP) and Table 5.7.4-1 (5QI 82).
- [2] R. Pan, P. Natarajan, F. Baker, G. White, *Proportional Integral Controller Enhanced (PIE)*, RFC 8033, 2017. <https://www.rfc-editor.org/rfc/rfc8033.html>
- [3] ITU-R M.2410, *Minimum requirements related to technical performance for IMT-2020 radio interface(s)*, 2017.
- [4] 3GPP TR 38.913, *Study on scenarios and requirements for next generation access technologies*.

Classical results we implemented

- [5] J. S. Kaufman, "Blocking in a shared resource environment," *IEEE Trans. Commun.* 29(10):1474–1481, 1981.
- [6] J. W. Roberts, "A service system with heterogeneous user requirements," in *Performance of Data Communication Systems and their Applications*, North-Holland, 1981 (book chapter).
- [7] P. B. Key, "Optimal control and trunk reservation in loss networks," *Probability in the Engineering and Informational Sciences* 4, 1990. Trunk reservation is optimal on a single link: B. L. Miller (1969), S. A. Lippman (1975).
- [8] D. Chiu, R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," *Computer Networks and ISDN Systems* 17, 1989 (AIMD).
- [9] P. Auer, N. Cesa-Bianchi, P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning* 47, 2002 (UCB1).
- [10] Y. Zhou, D. Chakrabarty, R. Lukose, "Budget constrained bidding in keyword auctions and online knapsack problems," WWW 2008 / WINE 2008. <https://archives.iw3c2.org/www2008/papers/pdf/p1243-zhou.pdf>
- [11] J. C. de Oliveira, C. Scoglio, I. F. Akyildiz, G. Uhl, "New preemption policies for DiffServ-aware traffic engineering to minimize rerouting in MPLS networks," *IEEE/ACM ToN* 12(4):733–745, 2004. <https://ianakyildiz.com/bwn/papers/2004/j10.pdf>

State of the art in network slicing we compared against

- [12] "Deep reinforcement learning for resource management on network slicing: a survey," *Sensors* 22(8):3031, 2022. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9032530/>

- [13] Wu et al., lightweight PPO-based slice admission (NSF PAR). <https://par.nsf.gov/servlets/purl/10343822>
- [14] "Machine learning for network slicing resource management: a comprehensive survey," 2020. <https://arxiv.org/pdf/2001.07974>
- [15] "Deep reinforcement learning for 6G AI-RAN: a comprehensive survey" (preprint). <https://arxiv.org/pdf/2608.14877>
- [16] A. Balasingam et al., "Zipper" (model-predictive control for vRAN slicing), USENIX NSDI 2024. <https://www.usenix.org/system/files/nsdi24-balasingam.pdf>
- [17] J. Liu, MPC-based resource allocation for uRLLC network slicing (doctoral thesis, 2024). <https://publica.fraunhofer.de/handle/publica/480889>
- [18] LACO: latency-driven network slicing orchestration with UCB, 2020. <https://arxiv.org/pdf/2009.03771>
- [19] DARIO: drift-aware policy selection for slice admission control (UCB). <https://boris.unibe.ch/199022>
- [20] FLAS: combined proactive and reactive autoscaling, 2025. <https://arxiv.org/pdf/2510.20388>
- [21] Traffic-aware threshold adjustment for NFV scaling using DRL, 2018. <https://arxiv.org/pdf/1811.08116>
- [22] D. Bertsimas et al., "Bounds and policies for loss networks." <https://www.mit.edu/~dbertsim/papers/Stochastic%20Networks/Bounds%20and%20policies%20for%20loss%20networks.pdf>

Notes on these sources. [10]: the competitive ratio $\ln(U/L)+1$ used in our pricing policy should be checked against the paper before it is quoted. [6] is a book chapter, not a journal paper. [12]–[22] were found through web search on 9 October 2026; several were read only as abstracts. All our measured numbers come from our own evidence files (section 12), not from these papers.

Phase 2b orchestrators: done, measured, and proven.

